QUERY BUILDER (Laravel)

- **Comprendre ce qu'est le Query Builder**
- **Faire des requêtes SELECT**
- **Utiliser WHERE**
- **Utiliser ORDER BY**
- **Utiliser GROUP BY**
- **Utiliser JOIN**
- **Insérer, modifier et supprimer des données**
- **Debugger une requête**

INTRODUCTION — C'EST QUOI LE QUERY BUILDER ?

Définition

Le **Query Builder** est :

Un outil intégré dans Laravel
qui permet d'écrire des requêtes SQL
de manière plus propre et sécurisée.

On ne tape pas du SQL brut directement.
Laravel construit la requête pour nous.

Classe utilisée

Le Query Builder utilise la façade :

Illuminate\Support\Facades\DB

Donc dans chaque fichier où on veut l'utiliser, on doit écrire en haut :

use Illuminate\Support\Facades\DB;

STRUCTURE D'UNE REQUÊTE QUERY BUILDER

Syntaxe générale :

```
DB::table('nom_table')
    ->condition()
    ->condition()
    ->action();
```

Exemple simple :

DB::table('clients')->get();

SQL équivalent :

SELECT * FROM clients;

OÙ ÉCRIRE LE CODE ?

Pour apprentissage débutant :

routes/web.php

Pourquoi ?

Parce que c'est plus simple pour tester rapidement.

SELECT (Lecture des données)

Sélection simple

Dans routes/web.php :

```
use Illuminate\Support\Facades\DB;

Route::get('/clients', function () {
    $clients = DB::table('clients')->get();
    foreach ($clients as $client) {
        echo $client->name . "<br>";
    }

});
```

Explication

use Illuminate\Support\Facades\DB;
→ On importe la façade DB

Route::get('/clients', function () {
→ On crée une route

DB::table('clients')
→ On sélectionne la table clients

->get()
→ On exécute la requête

$clients
→ Contient toutes les lignes sous forme d'objets

Résultat attendu

Dans le navigateur :

http://127.0.0.1:8000/clients

Affiche :

Ali
Karim
Sara

SELECT AVEC COLONNES SPÉCIFIQUES

```
$clients = DB::table('clients')
        ->select('name', 'email')
        ->get();
```

SQL équivalent :

SELECT name, email FROM clients;

Pourquoi ?

Pour éviter de récupérer des colonnes inutiles.

WHERE (Filtrer les données)

WHERE simple

```
$client = DB::table('clients')
        ->where('id', 1)
        ->get();
```

SQL :

SELECT * FROM clients WHERE id = 1;

Explication

where('id', 1)

'id' → colonne
1 → valeur
= est implicite

WHERE avec opérateur

```
$contracts = DB::table('contracts')
        ->where('amount', '>', 4000)
        ->get();
```

SQL :

SELECT * FROM contracts WHERE amount > 4000;

WHERE multiple

```
$clients = DB::table('clients')
        ->where('city', 'Casablanca')
        ->where('age', '>', 25)
        ->get();
```

SQL :

SELECT * FROM clients

WHERE city = 'Casablanca'

AND age > 25;

ORDER BY (Tri)

```
$clients = DB::table('clients')
        ->orderBy('name', 'asc')
        ->get();
```

asc → ordre croissant
desc → ordre décroissant

SQL :

SELECT * FROM clients ORDER BY name ASC;

GROUP BY (Regroupement)

```
$totals = DB::table('contracts')
        ->select('type', DB::raw('SUM(amount) as total'))
        ->groupBy('type')
        ->get();
```

SQL :

SELECT type, SUM(amount) as total

FROM contracts

GROUP BY type;

Pourquoi DB::raw ?

Parce que **SUM**() est une fonction SQL.
Laravel ne la connaît pas directement.

JOIN (Relation entre tables)

Supposons :

Table clients
Table contracts

```
$data = DB::table('contracts')
    ->join('clients', 'clients.id', '=', 'contracts.client_id')
    ->select('clients.name', 'contracts.type')
    ->get();
```

SQL :

```
SELECT clients.name, contracts.type
FROM contracts
JOIN clients
ON clients.id = contracts.client_id;
```

Explication

join('clients',
→ On relie la table clients

'clients.id'
→ clé primaire

'contracts.client_id'
→ clé étrangère

INSERT

```
DB::table('clients')->insert([
  'name' => 'Nadia',
  'email' => 'nadia@mail.com',
  'city' => 'Rabat'
]);
```

SQL :

```
INSERT INTO clients (name, email, city)
VALUES ('Nadia', 'nadia@mail.com', 'Rabat');
```

UPDATE

```
DB::table('clients')
  ->where('id', 1)
  ->update([
    'city' => 'Marrakech'
  ]);
```

SQL :

UPDATE clients

SET city = 'Marrakech'

WHERE id = 1;

DELETE

```
DB::table('clients')
    ->where('id', 3)
    ->delete();
```

SQL :

DELETE FROM clients WHERE id = 3;

DEBUG DES REQUÊTES

Voir la requête SQL générée

```
$query = DB::table('clients')->where('id', 1);
dd($query->toSql());
```

Affiche :

select * from `clients` where `id` = ?

Voir les valeurs réelles

```
dd($query->toSql(), $query->getBindings());
```

PAGINATION & MIGRATION

- **Pourquoi on utilise la pagination**
- **Comment afficher 5 éléments par page**
- **Comment Laravel crée automatiquement les pages**

PAGINATION

Définition

La pagination signifie :

--Diviser une grande liste en plusieurs petites pages.

Exemple réel :

Si on a 100 clients dans la base,
au lieu d'afficher 100 clients sur une seule page,
on affiche :

Page 1 → 5 clients
Page 2 → 5 clients
Page 3 → 5 clients

etc.

Pourquoi c'est important ?

Si on affiche tout :

- La page devient lente
- C'est difficile à lire
- Mauvaise expérience utilisateur

Exemple :

Dans routes/web.php

```
use Illuminate\Support\Facades\DB;

Route::get('/clients', function () {

   $clients = DB::table('clients')->paginate(5);

   return view('clients', compact('clients'));

});
```

Explication

use DB
→ On importe la classe DB

Route::get('/clients'
→ Création d'une route

DB::table('clients')
→ On sélectionne la table clients

->paginate(5)
→ On veut 5 clients par page

compact('clients')
→ On envoie les données vers la vue

Création de la vue

resources/views/clients.blade.php

```
<h2>Liste des clients</h2>

@foreach($clients as $client)

   {{ $client->name }} <br>

@endforeach


<br>

{{ $clients->links() }}
```

Explication

@foreach
→ Boucle sur les clients

$clients->links()
→ Laravel crée automatiquement :

Page 1 | Page 2 | Page 3

Résultat obtenu

Dans le navigateur :

http://127.0.0.1:8000/clients

Tu verras :

Ali
Karim
Sara
Nadia
Yassine

Puis en bas :

« **Previous** | 1 | 2 | 3 | **Next** »

Et l'URL change automatiquement :

? page=2

MIGRATIONS

- **C'est quoi une migration**
- **Comment créer une table**
- **Comment exécuter la migration**

Définition

Une migration est :

--- Un fichier PHP qui crée une table dans la base de données.

Au lieu d'aller dans phpMyAdmin
on crée les tables avec du code.

Pourquoi utiliser migration ?

- Travail en équipe
- Historique des modifications
- Versionnement
- Plus professionnel

Créer une migration

Dans le terminal :

php artisan make:migration create_clients_table

Laravel crée un fichier ici :

database/migrations/

Structure d'une migration simple

Ouvrir le fichier :

```
public function up()
{
    Schema::create('clients', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->integer('age');
        $table->timestamps();
    });
}
public function down()
{
    Schema::dropIfExists('clients');
}
```

Explication

Schema::create('clients')
→ Crée une table appelée clients

$table->id()
→ Colonne id auto-increment

$table->string('name')
→ Colonne texte

$table->integer('age')
→ Colonne nombre

$table->timestamps()
→ created_at et updated_at

Exécuter la migration

Dans terminal :

php artisan migrate

Résultat obtenu

Si tu ouvres phpMyAdmin,
tu verras une table :

clients

Avec colonnes :

id
name
email
age

created_at
updated_at

Annuler la migration

php artisan **migrate:rollback**

Cela supprime la table.

SEEDERS & FACTORIES

- Comment ajouter des données automatiquement
- Comment générer des fausses données

SEEDER

Définition

Un Seeder sert à :

-- Insérer des données automatiquement dans une table.

Créer un seeder

php artisan make:seeder ClientSeeder

Exemple

Dans **ClientSeeder.php** :

```
use Illuminate\Support\Facades\DB;

public function run()

{

   DB::table('clients')->insert([

      'name' => 'Ali',

      'email' => 'ali@mail.com',

      'age' => 25

     ]);
```

}

Lancer le seeder

Dans DatabaseSeeder.php :

$this->**call**(ClientSeeder::class);

Puis :

php artisan **db:seed**

Résultat obtenu

Dans phpMyAdmin
Un **client** nommé Ali est ajouté.

FACTORY

Une **factory** sert à créer facilement des données factices (fake) pour tester l'application.
Exemple : 50 clients générés automatiquement avec nom, email et âge aléatoires.

Créer une factory

Dans le terminal du projet :

php artisan make:factory ClientFactory

Explications :

- **make:factory** → On crée une factory
- **ClientFactory** → Nom de la factory

Résultat :
Fichier créé → database/factories/ClientFactory.php

Modifier la factory

Ouvrir **ClientFactory**.php et écrire ceci :

```
public function definition()
{
   return [
      'name' => fake()->name(),
      'email' => fake()->safeEmail(),
      'age' => fake()->numberBetween(18, 60),
   ];
}
```

Explication :

1. public function definition()
   → Méthode obligatoire pour définir les champs à générer.
2. return [...]
   → On retourne un tableau associatif avec les colonnes et leurs valeurs générées.
3. 'name' => fake()->name()
   → fake()->name() génère un nom aléatoire (ex : Ali, Karim…).
4. 'email' => fake()->safeEmail()
   → safeEmail() génère un email aléatoire sécurisé (ex : client@mail.com).
5. 'age' => fake()->numberBetween(18, 60)
   → numberBetween(18, 60) génère un nombre aléatoire entre 18 et 60.

Résultat attendu si on teste une seule fois :

```
[
 'name' => 'Ali Ben Ahmed',
 'email' => 'ali.ben@mail.com',
 'age' => 35
]
```

----Chaque exécution génère des données différentes.

Utiliser la factory dans DatabaseSeeder

Ouvrir database/seeders/**DatabaseSeeder**.php et ajouter :

\App\Models**Client::factory()->count(20)->create();**

Explication :

- **Client::factory()** → On appelle la factory du modèle Client
- **count(20)** → On veut 20 clients générés
- **create()** → Insère automatiquement dans la base de données

Exécuter le seeder

Dans le terminal :

php artisan db:seed

Résultat attendu

Dans phpMyAdmin :

- **20 lignes ajoutées dans la table clients**
- **Chaque ligne avec : nom aléatoire, email aléatoire, âge aléatoire**

Exemple dans la table clients :

id	name	email	age	created_at	updated_at
1	Ali Ben Ahmed	ali@mail.com	25	2026-02-18 12:00	2026-02-18 12:00
2	Sara Othman	sara.oth@mail.com	30	2026-02-18 12:00	2026-02-18 12:00
…	…	…	…	…	…